GUÍA DE PREPARACIÓN PREVIA

Conceptos fundamentales para el curso

Python y Analítica de Datos para Profesionales del Sector Financiero

Esta guía reúne los conceptos básicos que conviene conocer antes de la Sesión 1. No se requiere experiencia previa en programación: cada tema se explica con lenguaje sencillo y ejemplos del entorno bancario, y se relaciona con lo que se verá en el curso.

Cómo usar esta guía

- Leer los 11 temas con calma, idealmente antes de instalar el software del curso.
- Al final se incluyen preguntas de preparación: se espera una respuesta breve (3 a 5 líneas), redactada con las propias palabras del participante.
- Los fragmentos de código son solo ilustrativos; no es necesario memorizarlos ni ejecutarlos antes del curso.

1. ¿Qué es una hoja de cálculo?

Una hoja de cálculo es un programa que organiza la información en una cuadrícula de filas y columnas. Cada cruce entre una fila y una columna se llama celda, y puede contener un número, un texto, una fecha o una fórmula que calcula un resultado a partir de otras celdas. Los ejemplos más conocidos son Microsoft Excel, Google Sheets y LibreOffice Calc.

Elementos principales

- Celdas, filas y columnas: la unidad básica de almacenamiento y organización.
- Fórmulas y funciones: SUMA, PROMEDIO, SI, BUSCARV, entre otras.
- Herramientas de análisis: filtros, ordenamiento, tablas dinámicas y gráficos.

Fortalezas y límites

Su mayor virtud es la inmediatez: el usuario ve los datos y el resultado al instante, sin escribir código. Sus límites aparecen cuando el volumen o la repetición crecen: Excel admite un máximo de 1.048.576 filas por hoja, los archivos pesados se vuelven lentos, y los procesos manuales (copiar, pegar, aplicar fórmulas a mano) son difíciles de auditar y propensos a errores.

> *En el banco: la conciliación mensual de un reporte con cientos de miles de movimientos suele superar lo cómodo de manejar en una hoja de cálculo. Este es uno de los motivos por los que el curso presenta Python como una "hoja de cálculo programable".*

2. ¿Qué es un manejador de bases de datos relacional?

Una base de datos es una colección organizada de información que puede consultarse y actualizarse de forma eficiente. Un manejador (o sistema gestor) de bases de datos relacional, conocido como RDBMS, es el software que almacena esos datos, controla quién puede acceder a ellos y responde a las consultas de los usuarios y de las aplicaciones. Ejemplos: PostgreSQL, MySQL, Oracle Database y Microsoft SQL Server.

El modelo relacional

La información se organiza en tablas (también llamadas relaciones). Cada fila es un registro y cada columna es un campo. Las tablas se vinculan entre sí mediante claves:

- Clave primaria: campo que identifica de forma única cada registro (por ejemplo, el número de cliente).
- Clave foránea: campo que apunta a la clave primaria de otra tabla (por ejemplo, el número de cliente dentro de la tabla de transacciones).

Las consultas se escriben en SQL (Structured Query Language), un lenguaje especializado para pedirle datos a la base. Por ejemplo: SELECT * FROM transacciones WHERE monto > 5000;

Hoja de cálculo frente a base de datos relacional

Aspecto	Hoja de cálculo	Base de datos relacional
Volumen	Limitado y lento al crecer	Diseñada para millones de registros
Usuarios simultáneos	Difícil de coordinar	Múltiples usuarios con control de concurrencia
Integridad	Depende del cuidado del usuario	Reglas y claves que evitan datos inconsistentes
Seguridad	Control básico por archivo	Permisos por usuario, tabla y operación
Uso típico	Análisis puntual y reportes	Sistema de registro de la operación

> *En el curso se instala PostgreSQL, un manejador relacional de código abierto muy utilizado en la industria, y se verá cómo Python puede conectarse a él.*

3. ¿Qué es un lenguaje de programación?

Un lenguaje de programación es un conjunto de reglas y vocabulario que permite darle instrucciones precisas a una computadora. Un programa es una secuencia de esas instrucciones que resuelve un problema; la lógica paso a paso para resolverlo se denomina algoritmo. Una analogía útil es una receta de cocina: lista los ingredientes (datos) y los pasos (instrucciones) en un orden exacto.

Las computadoras solo ejecutan instrucciones binarias, de modo que los lenguajes actúan como un puente entre el lenguaje humano y el de la máquina. Se suele distinguir entre:

- Lenguajes de bajo nivel (como ensamblador): muy cercanos a la máquina, difíciles de leer y escribir.
- Lenguajes de alto nivel (como Python, Java o C#): más cercanos al lenguaje humano, más fáciles de aprender y mantener.

Existen además lenguajes especializados, como SQL para consultar bases de datos. Python es un lenguaje de propósito general: sirve para automatizar tareas, analizar datos, construir aplicaciones y mucho más.

4. ¿Qué es el lenguaje de programación Python?

Python es un lenguaje de programación de alto nivel, de propósito general y de código abierto. Fue creado por Guido van Rossum y publicado por primera vez en 1991. Se caracteriza por una sintaxis clara y legible, parecida a instrucciones en inglés sencillo, lo que lo hace una de las mejores opciones para quienes comienzan a programar.

Características principales

- Interpretado: el código se ejecuta línea por línea mediante un intérprete, sin un paso previo de compilación; esto permite probar ideas rápidamente.
- Gratuito y de código abierto: no requiere licencias y cuenta con una comunidad global muy activa.
- Multiplataforma: funciona en Windows, macOS y Linux.
- Gran ecosistema de librerías: paquetes listos para análisis de datos, visualización, automatización e inteligencia artificial.

Python en el sector financiero

Se utiliza para automatizar reportes, detectar anomalías y fraude, construir modelos de scoring crediticio, realizar conciliaciones masivas y analizar series de tiempo. De hecho, Pandas, la librería central de este curso, nació en 2008 dentro de un fondo de inversión cuantitativo para resolver problemas de análisis financiero.

Python incluye IDLE, un editor sencillo que se instala junto con el lenguaje y que se usará como entorno principal durante el curso.

5. Principales componentes del lenguaje Python

Todo programa en Python se construye combinando unos pocos componentes básicos. A continuación se describen con ejemplos breves.

5.1 Variables

Una variable es un nombre que se asocia a un valor guardado en la memoria, como una etiqueta puesta sobre una caja. Python reconoce el tipo del dato automáticamente: números enteros (int), decimales (float), texto (str) y valores lógicos verdadero/falso (bool).

```
saldo = 1500.75          # decimal
cliente = "Ana Pérez"    # texto
num_cuentas = 3          # entero
activo = True            # lógico
```

5.2 Estructuras de datos

Permiten guardar varios valores juntos en una sola variable. Las cuatro principales son:

Estructura	Descripción	Ejemplo
Lista	Colección ordenada y modificable; admite valores repetidos	montos = [120.5, 340.0, 89.9]
Tupla	Colección ordenada pero inmodificable	agencia = ("Caracas", "Sucursal 01")
Diccionario	Pares clave-valor; se accede por la clave	cliente = {"nombre": "Ana", "saldo": 1500.75}
Conjunto	Colección sin orden y sin elementos repetidos	paises = {"Venezuela", "Colombia", "Panamá"}

5.3 Estructuras de control condicional

Permiten que el programa tome decisiones: ejecuta unas instrucciones u otras según se cumpla o no una condición. Se escriben con if, elif y else. La sangría (espacios al inicio de la línea) es obligatoria y delimita el bloque.

```
if monto < 100:
    categoria = "bajo riesgo"
elif monto < 500:
    categoria = "riesgo medio"
else:
    categoria = "alto riesgo"
```

5.4 Estructuras de control repetitivas

Permiten repetir instrucciones sin escribirlas varias veces. El ciclo for recorre los elementos de una colección; el ciclo while repite mientras una condición siga siendo verdadera.

```
total = 0
for m in montos:
    total = total + m

while saldo > 0:
    saldo = saldo - 100
```

5.5 Funciones

Una función es un bloque de instrucciones con nombre que puede reutilizarse. Recibe datos de entrada (parámetros) y puede devolver un resultado con return. Evita repetir código y facilita su mantenimiento.

```
def clasificar(monto):
    if monto < 100:
        return "bajo riesgo"
    elif monto < 500:
        return "riesgo medio"
    else:
        return "alto riesgo"

print(clasificar(250))   # riesgo medio
```

5.6 Módulos

Un módulo es un archivo de Python (.py) que contiene funciones y definiciones listas para usar en otros programas. Se incorporan con la instrucción import. Python trae módulos incluidos (como statistics o math), y se pueden instalar librerías de terceros (como Pandas o Matplotlib), que son conjuntos de módulos.

```
import statistics
print(statistics.mean([120.5, 340.0, 89.9]))

import pandas as pd      # librería externa (requiere instalación)
```

6. ¿Qué es un entorno virtual en Python, cuál es su utilidad e importancia?

Un entorno virtual es una carpeta independiente que contiene su propia copia del intérprete de Python y su propio conjunto de librerías instaladas. Funciona como una caja de herramientas exclusiva para un proyecto: lo que se instala dentro no afecta al resto del equipo, ni lo que ya existe en el equipo afecta al proyecto.

¿Para qué sirve?

- Aislar proyectos: cada proyecto tiene sus propias librerías y versiones.
- Evitar conflictos: un proyecto puede requerir una versión de una librería y otro una distinta; sin entornos virtuales ambos chocarían.
- Mantener limpia la instalación principal de Python del equipo.

- Garantizar reproducibilidad: otra persona puede recrear exactamente el mismo entorno y obtener los mismos resultados, algo clave en un contexto bancario donde los procesos deben ser auditables.

Comandos básicos en Windows

```
python -m venv venv          # crear el entorno llamado "venv"
venv\Scripts\activate        # activarlo (aparece (venv) en la línea)
pip install pandas           # instalar librerías dentro del entorno
deactivate                   # salir del entorno
```

> *En el curso, las librerías (Pandas, Matplotlib, openpyxl y psycopg2-binary) se instalarán dentro de un entorno virtual. Por eso es indispensable que pip tenga acceso a pypi.org desde la red de cada participante.*

7. ¿Qué son las dependencias en Python?

Una dependencia es un paquete o librería externa que un programa necesita para funcionar. En lugar de escribir todo desde cero, el programador se apoya en código ya desarrollado y probado por la comunidad. Por ejemplo, un script que lee un archivo de Excel depende de la librería openpyxl.

Cómo se gestionan

- pip: la herramienta que descarga e instala paquetes (pip install nombre-del-paquete).
- PyPI (Python Package Index), en pypi.org: el repositorio oficial desde donde pip obtiene los paquetes.
- Dependencias indirectas: un paquete puede requerir otros. Por ejemplo, Pandas depende de NumPy, y pip instala ambos de forma automática.
- Versiones: cada paquete tiene un número de versión; conviene registrar cuáles se usaron.

Para dejar constancia de las dependencias de un proyecto se usa un archivo de texto llamado requirements.txt:

```
pip freeze > requirements.txt       # guardar la lista de paquetes
pip install -r requirements.txt     # reinstalarlos en otro equipo
```

Dependencias del curso

Paquete	Función
pandas	Manipulación y análisis de datos
matplotlib	Visualización de datos
openpyxl	Lectura y escritura de archivos Excel
psycopg2-binary	Conexión de Python con PostgreSQL

> *Buena práctica de seguridad: instalar paquetes únicamente desde fuentes oficiales (PyPI) y verificar bien el nombre antes de instalar, ya que existen paquetes maliciosos con nombres parecidos a los legítimos.*

8. Integración de Python con archivos, hojas de cálculo y bases de datos

Una de las mayores fortalezas de Python es que actúa como un puente entre fuentes de datos muy distintas. Un mismo programa puede leer un archivo, procesarlo y entregar el resultado en otro formato.

Archivos de texto y CSV

Un archivo CSV (valores separados por comas) es el formato de intercambio de datos más común: es un texto simple donde cada línea es un registro. Los datasets de Kaggle que se usan en el curso vienen en este formato.

```
import pandas as pd
df = pd.read_csv("clientes_banco.csv")     # leer
df.to_csv("resultado.csv", index=False)    # guardar
```

Hojas de cálculo (Excel)

Con la librería openpyxl, Pandas puede leer y escribir archivos de Excel, incluso con varias hojas, lo que permite automatizar reportes que antes se armaban a mano.

```
df = pd.read_excel("reporte.xlsx", sheet_name="Datos")
with pd.ExcelWriter("salida.xlsx") as w:
    df.to_excel(w, sheet_name="Datos", index=False)
    df.describe().to_excel(w, sheet_name="Resumen")
```

Bases de datos (PostgreSQL)

La librería psycopg2 permite que Python se conecte a PostgreSQL, ejecute consultas SQL y traiga los resultados a un DataFrame de Pandas para analizarlos.

```
import psycopg2
import pandas as pd

conexion = psycopg2.connect(host="localhost", dbname="banco",
                            user="postgres", password="su_clave")
df = pd.read_sql("SELECT * FROM transacciones", conexion)
conexion.close()
```

> *Al usar read_sql con una conexión de psycopg2, las versiones recientes de Pandas pueden mostrar una advertencia (UserWarning) que recomienda usar SQLAlchemy. El código funciona igualmente; es solo un aviso. En un entorno real, la clave nunca se escribe directamente en el código.*

Flujo típico de integración

Origen	Procesamiento en Python	Destino
Archivo CSV o base de datos PostgreSQL	Limpieza, filtros, agrupaciones, estadística (Pandas)	Reporte en Excel o gráfico (openpyxl, Matplotlib)

9. ¿Qué es la librería Pandas y para qué sirve?

Pandas es la librería de código abierto más usada en Python para manipular y analizar datos en forma de tablas. Fue creada en 2008 por Wes McKinney y está construida sobre NumPy. Su estructura principal es el DataFrame, una tabla con filas y columnas etiquetadas equivalente a una hoja de Excel, y la Serie (Series), que representa una sola columna.

¿Para qué sirve?

- Cargar datos desde CSV, Excel, bases de datos y otros formatos.
- Explorar rápidamente un dataset: head(), info(), describe().
- Filtrar, ordenar, agrupar y combinar tablas.
- Limpiar datos: manejar nulos, duplicados y formatos.
- Calcular estadísticas descriptivas y exportar resultados.

Equivalencias con Excel

En Excel	En Pandas
Abrir un archivo	pd.read_csv() / pd.read_excel()
Autofiltro	df[df["saldo"] > 1000]
Ordenar	df.sort_values("saldo")
Tabla dinámica	df.groupby("geografia")["saldo"].mean()
PROMEDIO, SUMA	df["saldo"].mean(), df["saldo"].sum()
BUSCARV	pd.merge(tabla1, tabla2, on="id_cliente")

> *Regla de oro del curso: nunca imprimir un DataFrame completo con print(df); usar df.head(), df.sample(10) y df.shape para no saturar la consola.*

10. ¿Qué es la librería Matplotlib y para qué sirve?

Matplotlib es la librería estándar de Python para crear gráficos. Su módulo más utilizado es pyplot, que permite construir una figura paso a paso: elegir el tipo de gráfico, agregar título, etiquetas de ejes y leyenda, y finalmente mostrarla en pantalla o guardarla como imagen. Fue creada por John D. Hunter a comienzos de la década del 2000 y funciona en conjunto con Pandas.

Gráficos más usados en el curso

Tipo	Para qué se usa	Ejemplo bancario
Barras	Comparar categorías	Monto total de transacciones por sucursal
Líneas	Ver la evolución en el tiempo	Transacciones por mes
Histograma	Ver cómo se distribuye una variable numérica	Distribución de montos de transacciones

```
import matplotlib.pyplot as plt

plt.hist(df["monto"], bins=50)
plt.title("Distribución de montos")
plt.xlabel("Monto")
plt.ylabel("Cantidad de transacciones")
plt.savefig("histograma.png")   # guardar como imagen
plt.show()                      # abrir ventana
```

11. Conceptos estadísticos

La estadística descriptiva resume y describe un conjunto de datos con pocos números. En el curso estos conceptos se introducen de forma progresiva; esta sección es una primera aproximación para familiarizarse con el vocabulario.

Concepto	Qué es	Ejemplo bancario
Media	Promedio: suma de los valores dividida entre la cantidad de datos	Saldo promedio de los clientes
Mediana	Valor central al ordenar los datos de menor a	Saldo “típico” de un cliente

Concepto	Qué es	Ejemplo bancario
	mayor; la mitad queda por debajo y la mitad por encima	
Moda	Valor que más se repite	Categoría de riesgo más frecuente
Desviación estándar	Mide qué tan dispersos están los datos alrededor de la media	Qué tan parecidos o distintos son los saldos de una sucursal
Percentiles	Valores que dividen los datos ordenados en 100 partes; el percentil 90 deja por debajo al 90% de los datos	Monto bajo el cual cae el 90% de las transacciones
Cuartiles	Percentiles 25, 50 y 75: dividen los datos en cuatro partes iguales	Dividir a los clientes en cuatro grupos según su saldo
Outliers	Valores atípicos, muy alejados del resto; pueden ser errores o casos legítimos	Una transacción de un monto extraordinariamente alto
Distribución	Forma en que se reparten los valores: dónde se concentran y si hay colas largas	Muchas transacciones pequeñas y pocas muy grandes
Correlación	Mide la relación entre dos variables, entre -1 y 1; no implica causalidad	Relación entre monto y probabilidad de fraude

Ejemplo resuelto

Sueldos mensuales (en USD) de seis empleados de una sucursal: 800, 850, 900, 900, 950 y 10.000 (este último corresponde al gerente).

Medida	Resultado	Interpretación
Media	2.400	El promedio se infla por el valor extremo; no representa a la mayoría
Mediana	900	Refleja mejor el sueldo típico
Moda	900	Es el sueldo que más se repite
Desviación estándar	≈ 3.724	Alta dispersión, causada por el valor de 10.000
Cuartiles (Q1 y Q3)	862,5 y 937,5	La mitad central de los datos está entre estos dos valores
Percentil 90	5.475	Solo el 10% de los datos queda por encima de este valor
Outlier	10.000	Supera por mucho el límite Q3 + 1,5 × (Q3 − Q1) = 1.050

Este ejemplo ilustra por qué en análisis de datos nunca basta con mirar solo el promedio: media y mediana pueden contar historias muy distintas.

Cómo se calculan en Pandas

```
df["saldo"].mean()                    # media
df["saldo"].median()                  # mediana
df["saldo"].mode()                    # moda
df["saldo"].std()                     # desviación estándar
df["saldo"].quantile([0.25, 0.5, 0.75, 0.9])   # cuartiles y percentil 90
df[["monto", "fraude"]].corr()         # correlación
```

> *Interpretación de la correlación: un valor cercano a 1 indica relación positiva fuerte (si una variable sube, la otra tiende a subir); cercano a -1, relación negativa fuerte; cercano a 0, ausencia de relación lineal.*

Preguntas de preparación

Responder cada pregunta con 3 a 5 líneas, con palabras propias y, cuando sea posible, con un ejemplo del trabajo diario. No se busca una respuesta técnica perfecta, sino demostrar comprensión.

1. ¿Qué es una hoja de cálculo y cuáles son sus límites cuando el volumen de datos crece?
2. ¿Qué es un manejador de bases de datos relacional y en qué se diferencia de una hoja de cálculo?
3. ¿Qué es un lenguaje de programación?
4. ¿Qué es Python y por qué se utiliza en el sector financiero?
5. ¿Cuáles son los principales componentes del lenguaje Python? Dar un ejemplo de cada uno.
6. ¿Qué es un entorno virtual y por qué es importante usarlo?
7. ¿Qué es una dependencia en Python y cómo se instala?
8. ¿De qué manera Python se integra con archivos, hojas de cálculo y bases de datos?
9. ¿Qué es Pandas y para qué sirve?
10. ¿Qué es Matplotlib y para qué sirve?
11. Explicar con un ejemplo la diferencia entre media y mediana, y qué es un outlier.